

Especificacion del Lenguaje Summarize

Proposito

Summarize es un lenguaje compacto para escribir algoritmos numericos y de control que puedan traducirse de forma directa al codigo intermedio FIS-25. Su nombre viene de su proposito principal: resumir instrucciones largas de bajo nivel en instrucciones fuente cortas, legibles y rapidas de escribir.

La idea es que el programador escriba `wh`, `fn`, `ret`, `T`, `F`, expresiones aritmeticas y llamadas nativas simples, mientras el compilador genera las etiquetas, temporales, saltos y operaciones de tres direcciones que exige FIS-25.

Paradigma

El lenguaje es imperativo y procedural:

- El estado se modela con variables mutables.
- El flujo usa condicionales, ciclos `wh` y ciclos `for`.
- Las funciones permiten agrupar calculos reutilizables.
- Los bloques se delimitan por indentacion, no por llaves.

Extension y formato

La extension recomendada es `.summ`.

Los comentarios empiezan con `#`. Las lineas vacias se ignoran.

```
# comentario
int x = 3
wh x > 0:
    x = x - 1
```

Tipos

Tipo	Uso
<code>int</code>	Enteros con signo para contadores, indices y aritmetica exacta.
<code>float</code>	Numeros reales para aproximaciones y divisiones con precision.
<code>bool</code>	Valores logicos <code>T</code> y <code>F</code> . En FIS-25 se emiten como <code>1</code> y <code>0</code> .
<code>str</code>	Cadenas literales, principalmente para <code>print</code> .
<code>var</code>	Inferencia explicita a partir de una expresion inicial.

Reglas relevantes:

- `int` se puede promover a `float`.

- `float` no se asigna implícitamente a `int`.
- Las condiciones de `if`, `elif` y `wh` deben ser `bool`.
- Los límites de `for i in a..b` deben ser `int`.
- Los literales numéricos no aceptan ceros a la izquierda: `01` es inválido.

Palabras reservadas

```
if elif else wh for in fn ret var
int float str bool T F
```

Operadores

Categoría	Operadores
Aritmética	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>
Comparación	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>
Lógica	<code>&</code> , <code>^</code>
Asignación	<code>=</code>
Rango	<code>..</code>

La precedencia sigue el orden usual: paréntesis, unarios, multiplicativos, aditivos, comparaciones, `&`, `|`.

Declaraciones y asignaciones

```
int x = 10
float y = x + 2.5
bool ok = x < 20
str msg = "done"
var inferred = y / 2.0
```

```
x = x + 1
```

También se permite inferencia en la primera asignación:

```
count = 0
```

Para el backend FIS-25 se recomienda no usar shadowing de nombres, porque la memoria intermedia es plana y global.

Control de flujo

```
if x < y:
    print(x)
elif x == y:
    print("same")
else:
    print(y)
```

```
wh n > 0:
    n = n - 1
```

El ciclo **for** usa limite final exclusivo:

```
for i in 0..10:
    print(i)
```

Ese ejemplo imprime valores de **0** a **9**.

Funciones

```
fn sum(int a, int b) -> int:
    ret a + b

int result = sum(2, 3)
print(result)
```

Reglas:

- Las funciones deben declararse antes de llamarse.
- El numero y tipo de argumentos se valida semanticamente.
- **ret** solo es valido dentro de una funcion.
- El tipo de retorno debe coincidir con la firma.

Funciones nativas

Funcion	Tipo	Traduccion FIS-25
print(x)	int tecnico	PRINT <x>
input()	int	INPUT <dest>
pixel(x, y, c)	int tecnico	PIXEL <x> <y> <c>
key(k)	bool	KEY <k> <dest>

El mapeo de `key(k)` sigue la nota del simulador: 0=Up, 1=Down, 2=Left, 3=Right, 4=W, 5=S, 6=A, 7=D, 8=Space.

Ejemplo completo

```
int n = input()
int steps = 0

wh n > 1:
    if n % 2 == 0:
        n = n / 2
    else:
        n = 3 * n + 1
        steps = steps + 1

print(steps)
```

Compilacion:

```
./summc -o collatz.fis < examples/collatz.summ
```